Documentação de entrega — Software S&OP Manus

Data de referência: 6 de maio de 2026
Audiência: novo desenvolvedor assumindo o projeto

---

1. Visão geral do repositório

Este diretório agrupa material de **Sales & Operations Planning (S&OP)** para o contexto Manus / Printbag: referências (Easy 360), dados auxiliares em CSV, scripts Python legados de previsão de demanda e **o aplicativo principal**, que vive na pasta `sop-system/`.

Pasta / arquivo	Função
`sop-system/`	**Código-fonte da aplicação web** (frontend React + backend Node + MariaDB). É o núcleo técnico a manter.
`package.json` (raiz)	Workspace npm com atalhos (`dev`, `install:app`, `db:local:start`, etc.) que delegam para `sop-system`.
`1.Software Easy Informações/`	Documentação e prints de referência do produto Easy 360 (contexto de negócio, não é código executável do SOP atual).
`3. Script Python Previsão Demanda/`	Scripts e planilhas históricas de previsão (Excel + Python); complementares ao fluxo moderno em `sop-system`.
`dados sistema/`	CSVs de cadastros auxiliares (de-para, dias úteis, fluxos, linhas, etc.) usados como base ou importação.
`Exemplos de exel scrip/`	Exemplos de saídas Excel.
`Mapeamento Software S&OP Printbag.docx`	Documento de mapeamento funcional (negócio).

---

2. O que é o sistema (`sop-system`)

É uma **aplicação corporativa de S&OP** com:

- **Cadastros** de famílias, subfamílias, produtos, clientes, recursos, calendário (dias úteis), roteiros, paradas, marcas, gestores, de-para de códigos, fluxos, turnos e tabelas auxiliares (status, tipo de negócio, tipo de estoque, linhas, modelos, tamanhos).
- **Demanda:** dashboards, acurácia, previsões, colaborações, consensos, orçamentos.
- **Capacidade:** dashboard e cálculo de capacidade.
- **Vendas:** histórico, carteira de pedidos, forecast, importação de faturamento.
- **Produção:** metas de produção.

- **Integrações:** SharePoint e jobs de **previsão via Python** (execução externa orquestrada pelo backend).
- **Administração:** usuários, marcas, níveis de acesso (gestor / gerente / master).

Autenticação prevê **OAuth** em produção; em desenvolvimento local usa **bypass** configurável via variáveis de ambiente (ver secção 6).

---

3. Stack técnica

3.1 Frontend

Tecnologia	Uso
React 19	UI
TypeScript	Tipagem
Vite 7	Build e dev server integrado ao Express em modo middleware
Wouter	Roteamento no cliente
TanStack React Query	Cache e chamadas assíncronas
tRPC (client + react-query)	Chamadas type-safe ao backend
Tailwind CSS 4	Estilização
Radix UI + shadcn-style components	Componentes acessíveis
Recharts	Gráficos
React Hook Form + Zod	Formulários e validação
Framer Motion	Animações
date-fns, **xlsx**, etc.	Utilitários e Excel

3.2 Backend

Tecnologia	Uso
Node.js (ESM)	Runtime (`"type": "module"` no `package.json`)
Express 4	HTTP server
tRPC 11	API tipada em `/api/trpc`
SuperJSON	Serialização
Drizzle ORM	Acesso ao MySQL/MariaDB
mysql2	Driver
Zod	Validação de inputs nos routers

Tecnologia	Uso
Multer	Uploads
AWS SDK S3	Armazenamento/referência S3 (quando configurado)
jose	JWT / sessão
dotenv	Variáveis de ambiente

3.3 Base de dados

- **MySQL compatível / MariaDB** (localmente MariaDB portátil na porta **3307**, usuário `root` / senha `root`, database `sop_system`).
- Migrações e schema em `sop-system/drizzle/` (`schema.ts`, SQL gerados, `drizzle-kit`).

3.4 Previsão (Python)

- Ambiente virtual recomendado: `sop-system/.venv-previsao/`.
- Dependências em `sop-system/requirements-previsao-python.txt`: NumPy, Pandas, scikit-learn, openpyxl, python-dateutil, matplotlib, statsmodels.
- O servidor Node invoca o interpretador configurado em **`PREVISAO_PYTHON_CMD`** (caminho para `python.exe` da venv).

3.5 Ferramentas de desenvolvimento

- **pnpm** (gerenciador declarado no `package.json` do app).
- **tsx** — execução/watch do servidor TypeScript.
- **esbuild** — bundle do servidor para produção.
- **Vitest** — testes.
- **Prettier** — formatação.
- **PowerShell** — scripts de setup local e MariaDB portátil no Windows.

---

4. Estrutura principal de `sop-system`

```
sop-system/
├── client/              # SPA React (entrada: client/index.html, src/)
├── server/
│   ├── _core/           # Express, Vite middleware, OAuth, contexto tRPC
│   ├── routers.ts       # Router tRPC principal (milhares de linhas — domínio completo)
│   ├── db.ts            # Funções de persistência
│   ├── previsaoPython.ts
│   ├── sharepoint.ts
│   └── ...
├── shared/              # Tipos e constantes compartilhadas
├── drizzle/             # Schema Drizzle + migrações SQL
├── scripts/             # setup-local-dev.ps1, start-local-db.ps1, importações, etc.
```

```
├── patches/             # Patch pnpm (ex.: wouter)
├── dist/                # Saída de build (produção)
├── .env / .env.example
├── vite.config.*
└── package.json
```

Entrada do servidor: `server/_core/index.ts` — monta Express, OAuth em `/api/oauth/callback`, tRPC em `/api/trpc`, e em desenvolvimento o **Vite em middleware**; em produção serve estáticos de `dist/public`.

---

5. O que já foi implementado (domínio na API)

O arquivo `server/routers.ts` expõe routers tRPC agrupados por domínio, incluindo (lista não exaustiva dos grupos principais):

`system`, `auth`, `familias`, `subfamilias`, `produtos`, `clientes`, `recursos`, `diasUteis`, `hierarquias`, `membros`, `paradas`, `roteiros`, `vendas`, `carteiraPedidos`, `forecasts`, `orcamentos`, `previsoes`, `colaboracoes`, `consensos`, `metasProducao`, `faturamento`, cadastros auxiliares (`statusCadastro`, `tiposNegocio`, `tiposEstoque`, `linhas`, `modelos`, `tamanhos`), `marcas`, `gestores`, `gestorMarca`, `deParaCodigos`, `sharepoint`, `previsaoPython`, `niveisAcesso`, `usuarios`, `turnos`, `recursosMensais`, `fluxos`, `cadastrosSistema`, `dashboard`, `capacidade`, `pesquisa`.

O schema Drizzle (`drizzle/schema.ts`) modela usuários (com **nível de acesso** 1–3), relacionamento usuário–marca, famílias/subfamílias, produtos com diversos atributos comerciais e operacionais, e demais entidades alinhadas aos cadastros.

---

6. Configuração e execução local (Windows)

6.1 Pré-requisitos

- Node.js compatível com as versões do `package.json`.
- PowerShell para scripts `.ps1`.
- Opcional: Python 3 para criar `.venv-previsao` (previsão).

6.2 Variáveis de ambiente

Copiar `sop-system/.env.example` para `sop-system/.env` e ajustar:

- **`DATABASE_URL`** — ex.: `mysql://root:root@127.0.0.1:3307/sop_system`
- **`PORT`** / **`HOST`** — servidor HTTP (padrão típico porta **3012**)
- **`DEV_AUTH_BYPASS=true`** — desenvolvimento sem OAuth real
- **`PREVISAO_PYTHON_CMD`** — caminho absoluto para `python.exe` da venv de previsão

6.3 Scripts npm (raiz do monorepo leve)

Na pasta **`Software SOP Manus`** (raiz):

Comando	Efeito
`npm run install:app`	Instala dependências em `sop-system`
`npm run setup:local`	Sobe MariaDB local, aplica schema Drizzle se necessário, seed básico
`npm run setup:python-previsao`	Cria venv e instala requirements de previsão
`npm run db:local:start`	Apenas inicia MariaDB portátil (3307)
`npm run dev`	MariaDB + servidor de desenvolvimento (watch)

Equivalentes diretos dentro de `sop-system`: ver `package.json` scripts.

6.4 Banco local portátil

- Script: `sop-system/scripts/start-local-db.ps1`.
- Baixa/extrai MariaDB 11.8 winx64 em `sop-system/.local/`, dados em `sop-system/.local/mariadb-data/`.
- **Refatoração importante:** caminhos são calculados a partir de **`$PSScriptRoot`** (não dependem do nome da pasta pai do projeto); `my.ini` é regenerado para evitar caminhos absolutos obsoletos após renomear pastas.
- Porta **3307**; cria database `sop_system` se aplicável (via fluxo de setup).

6.5 Build de produção

No `sop-system`:

- `pnpm run build` — Vite (`dist/public`) + bundle do servidor (`dist/index.js`).
- `pnpm run start` — executa `NODE_ENV=production node dist/index.js` (ajuste de env no Windows conforme necessidade).

---

7. Rotas da interface (wouter)

Principais grupos de URLs (ver `client/src/App.tsx`):

- `/` — Home
- **Demanda:** `/demanda/dashboard`, `acuracidade`, `previsoes`, `colaboracoes`, `consensos`, `orcamentos`
- **Capacidade:** `/capacidade/dashboard`, `/capacidade/calculo`
- **Produção:** `/producao/metas`
- **Vendas:** `/vendas/*` (dashboard, histórico, carteira, forecast, importar)
- **Cadastros:** `/cadastros/*`
- **Integração:** `/integracao/sharepoint`, `/previsao-python`, jobs de visualização
- **Admin:** `/admin/usuarios`, `/admin/marcas`

---

8. Pontos de atenção para o novo desenvolvedor

1. `server/routers.ts` **é muito grande** — para alterações, buscar o router do domínio (ex.: `previsaoPython:`) e extrair lógica para `server/services/` quando fizer sentido.
2. **Segredos** — não commitar `.env` ; usar `.env.example` como referência.
3. **Previsão Python** — tempo limite e heartbeat configuráveis (`PREVISAO_PYTHON_TIMEOUT_MIN` , `PREVISAO_PYTHON_HEARTBEAT_SEC`).
4. **Renomeação de pastas** — o projeto já foi renomeado (ex.: remoção de `&` no nome da pasta); scripts de DB locais foram ajustados para **não fixar caminhos absolutos**.
5. **pnpm patches** — há patch aplicado a `wouter` ; manter ao atualizar dependências.

---

9. Referências rápidas de arquivos

Arquivo	Descrição
`sop-system/package.json`	Scripts e dependências
`sop-system/server/_core/index.ts`	Bootstrap Express + tRPC + Vite
`sop-system/server/routers.ts`	API tRPC
`sop-system/drizzle/schema.ts`	Modelo de dados
`sop-system/scripts/start-local-db.ps1`	MariaDB portátil
`sop-system/scripts/setup-local-dev.ps1`	Setup inicial do BD + seed
`sop-system/.env.example`	Template de configuração

Documento gerado para facilitar a continuidade do desenvolvimento do Software S&OP Manus.